



COMSATS University Islamabad

Ecommerce Inventory Management System

Midterm Project Report

Submitted by:

Ayesha (FA23-BCS-128)

Urva tul Wusqa (FA23-BCS-101)

Rameesha Akram (FA23-BCS-087)

Submitted to:

Dr Basit Raza

Date of Submission:

April 10, 2025

Table of Contents

Overview:	5
Dataset Details	5
Entity Relationship Diagram(ERD):	5
Collections:	6
1. Products Collection	6
2. Customers Collection	6
3. Orders Collection	6
4. Payments Collection	7
5. Shipping Collection	7
6. Reviews Collection	8
7. Categories Collection	8
8. Suppliers Collection	8
9. Cart Collection	8
Key Relationships:	9
Improvements in previous Database:	10
Database Indexing:	10
► Purpose:	10
► Implemented Indexes	10
► Conclusion:	12
Model Implementation and Testing	13
TRANSACTION:	13
Introduction:	13
Concurrency Control	19
Identify Collections for Locking	19
Queries that add version field in collections:	20
Optimistic Locking in Cart Updates:	21
Optimistic locking in product updates:	22
Quires that add fields for locking in Orders and Payment collections	23
Create Views in MongoDB Shell	25
1. View: Product Summary	25
2. View: Order Summary (hide sensitive data)	26
3. View : Customers View (hide phone, show address)	27

4. View: Payments View (show paid only).....	27
5. View: Reviews View (high rating only)	28
6. View: Suppliers View (summary only) as "SupplierInfoView"	29
7. View: Shipping View (in-transit only) as "ActiveShipmentsView"	29
8. View: Carts View (show only cart totals) as "CartSummaryView"	29
Successful query after creating views for Reviews View (high rating only).....	29
Materialized View.....	29
1. View: Monthly Sales Summary	29
2. View: Top-Spending Customers (from Orders).....	30
3. View: Frequent Payment Methods (from Payments).....	31
4. View: Most Reviewed Products (from Reviews) as "ProductReviewStats"	31
5. View: Shipping Delay Report (from Shipping) as "ShippingDelayReport"	31
6. View: Inventory Summary by Category (from Products) as "CategoryInventorySummary"	31
Simulate Access Control (No User Management in Current Version)	31
NoSQL Injection.....	32
1. Identify Injection Risk Points	32
2. Test the Attack (to Understand the Risk).....	32
3. Replace with Strict Filtered Query.....	32
4. Ban the Use of \$where, \$function (Dangerous).....	33
5. Protect Admin Filters (Querying with Inputs)	34
6. Disable JavaScript Operators in Queries.....	34
7. Final Secure Query Examples.....	34
User Manual for Graphic User interface:.....	36
Project Setup	36
Login Window:	36
Authorized Users:	36
Menu Window:	37
Create Menu:.....	38
Read Menu:	39
Update menu:	40
Delete Menu:.....	42
Queries menu:	43
Show Summary Statistics:	45

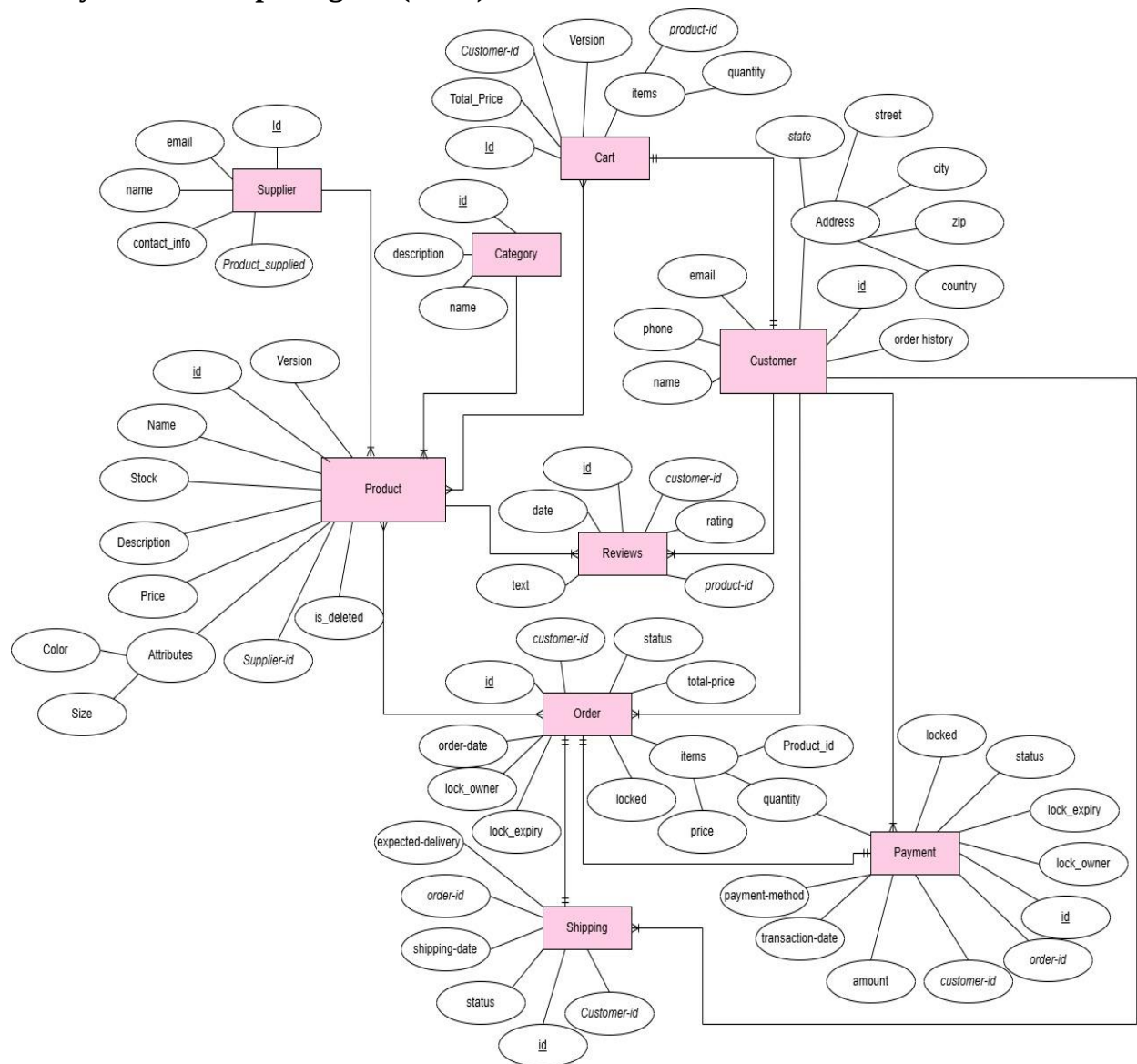
Chart Visualisation:	46
Challenges and Solutions.....	47

Overview:

The Ecommerce Inventory & Order Management System is designed to streamline inventory tracking, order processing, and customer management for online businesses. Built on MongoDB, the system leverages advanced database techniques such as concurrency control, transactions, views, materialized views, and security measures to ensure efficiency, scalability, and data integrity. This report documents the implementation, GUI design, and enhancements made since the midterm submission.

Dataset Details

Entity Relationship Diagram(ERD):



Collections:

The system uses MongoDB collections with the following schema:

1. Products Collection

Fields:

```
{_id: ObjectId,  
name: String,  
description: String,  
price: Number,  
stock: Number,  
category_id: ObjectId,  
supplier_id: ObjectId,  
attributes: {  
  color: String,  
  size: String}  
}
```

2. Customers Collection

Fields:

```
{_id: ObjectId,  
name: String,  
email: String,  
phone: String,  
order_history: [ObjectId],  
address: {  
  street: String,  
  city: String,  
  country: String }  
}
```

3. Orders Collection

Fields:

```
{ _id: ObjectId,  
  customer_id: ObjectId,  
  items: [{  
    product_id: ObjectId,  
    quantity: Number,  
    price: Number }],  
  total_price: Number,  
  status: String,  
  order_date: ISODate  
}
```

4. Payments Collection

Fields:

```
{_id: ObjectId,  
  order_id: ObjectId,  
  customer_id: ObjectId,  
  amount: Number,  
  payment_method: String,  
  status: String,  
  transaction_date: ISODate  
}
```

5. Shipping Collection

Fields:

```
{ _id: ObjectId,  
  order_id: ObjectId,  
  customer_id: ObjectId,  
  shipping_date: ISODate,  
  expected_delivery: ISODate,  
  status: String  
}
```

6. Reviews Collection

Fields:

```
{_id: ObjectId,  
  product_id: ObjectId,  
  customer_id: ObjectId,  
  rating: Number,  
  review_text: String,  
  review_date: ISODate  
}
```

7. Categories Collection

Fields:

```
{_id: ObjectId,  
  name: String,  
  description: String  
}
```

8. Suppliers Collection

Fields:

```
{_id: ObjectId,  
  name: String,  
  contact_info: String,  
  products_supplied: [ObjectId]  
}
```

9. Cart Collection

Fields:

```
{_id: ObjectId,  
  customer_id: ObjectId,  
  items: [{  
    product_id: ObjectId,  
    quantity: Number}],
```

total_price: Number

}

Key Relationships:

Relationship	Type	Collections Involved
Customers ↔ Cart	One-to-One	Customers ↔ Cart
Customers → Orders	One-to-Many	Customers → Orders
Customers → Payments	One-to-Many	Customers → Payments
Customers → Reviews	One-to-Many	Customers → Reviews
Customers → Addresses	One-to-Many	Customers → Addresses
Orders ↔ Payments	One-to-One	Orders ↔ Payments
Orders ↔ Shipping	One-to-One	Orders ↔ Shipping
Orders ↔ Products	Many-to-Many	Orders ↔ Products
Products → Categories	Many-to-One	Products → Categories
Products → Suppliers	Many-to-One	Products → Suppliers
Products ↔ Reviews	One-to-Many	Products ↔ Reviews
Products ↔ Cart	Many-to-Many	Products ↔ Cart
Suppliers → Products	One-to-Many	Suppliers → Products

Fields	Index Type	Purpose	Command
--------	------------	---------	---------

customer_id	Single Field	Retrieve all orders for a customer	db.orders.createIndex({ customer_id: 1 })
items.product_id	Multikey	Analyze which orders include a certain product	db.orders.createIndex({ "items.product_id": 1 })
order_date	Single Field	Sort orders by most recent date	db.orders.createIndex({ order_date: -1 })
status	Single Field	Filter orders based on current status	db.orders.createIndex({ status: 1 })
customer_id + order_date	Compound	Retrieve recent orders of a specific customer	db.orders.createIndex({ customer_id: 1, order_date: -1 })

Create Refresh

VIEWINGINDEXESSearch Indexes

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	36.9 KB	39 (since Mon May 19 2025)	UNIQUE	READY
> Orders by Customer ID	REGULAR	36.9 KB	0 (since Mon May 19 2025)	COMPOUND	READY
> Orders By status	REGULAR	36.9 KB	0 (since Mon May 19 2025)	COMPOUND	READY
> items.product_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> order_date_-1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY

4. Reviews Collection

Fields	Index Type	Purpose	Command
product_id	Single Field	Retrieve reviews for a product	db.reviews.createIndex({ product_id: 1 })
customer_id	Single Field	Find reviews posted by a specific customer	db.reviews.createIndex({ customer_id: 1 })
product_id + rating	Compound	Display top-rated or filtered reviews	db.reviews.createIndex({ product_id: 1, rating: -1 })

Create Refresh

VIEWINGINDEXESSearch Indexes

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	36.9 KB	4 (since Mon May 19 2025)	UNIQUE	READY
> customer_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> product_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> product_id_1_rating_-1	REGULAR	20.5 KB	0 (since Wed May 28 2025)	COMPOUND	READY

5. Payments Collection

Fields	Index Type	Purpose	Command
order_id	Single Field	Match payments to orders	db.payments.createIndex({ order_id: 1 })
customer_id	Single Field	Track payments by customer	db.payments.createIndex({ customer_id: 1 })
status	Single	Identify pending/paid	db.payments.createIndex({ status: 1 })

	Field	payments	
customer_id + status	Compound	Filter payments by customer and status	db.payments.createIndex({ customer_id: 1, status: 1 })

CreateRefresh

VIEWINGINDEXESSearch Indexes

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	36.9 KB	13 (since Mon May 19 2025)	UNIQUE	READY
> Payments by Order ID	REGULAR	36.9 KB	0 (since Mon May 19 2025)	UNIQUE	READY
> Payments by Methods	REGULAR	36.9 KB	0 (since Mon May 19 2025)		READY
> customer_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> status_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> customer_id_1_status_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)	COMPOUND	READY

6. Shipping Collection

Fields	Index Type	Purpose	Command
order_id	Single Field	Track shipping for a specific order	db.shipping.createIndex({ order_id: 1 })
customer_id	Single Field	Retrieve shipments for a customer	db.shipping.createIndex({ customer_id: 1 })
Status	Single Field	Track shipping progress	db.shipping.createIndex({ status: 1 })
expected_delivery	Single Field	Retrieve based on delivery date	db.shipping.createIndex({ expected_delivery: 1 })
customer_id + status	Compound	Identify active or pending shipments	db.shipping.createIndex({ customer_id: 1, status: 1 })

CreateRefresh

VIEWINGINDEXESSearch Indexes

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	36.9 KB	13 (since Mon May 19 2025)	UNIQUE	READY
> Shipments by Expected Delivery Date	REGULAR	36.9 KB	7 (since Mon May 19 2025)		READY
> order_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> customer_id_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> status_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)		READY
> customer_id_1_status_1	REGULAR	20.5 KB	0 (since Wed May 28 2025)	COMPOUND	READY

► Conclusion:

The implementation of indexes significantly optimized query response times across all major operations in the database. Multikey indexes helped with accessing arrays efficiently (like items in carts or orders), while compound indexes improved performance on common combined queries, such as fetching recent customer orders or filtering reviews by rating.

This indexing strategy ensures the e-commerce system can handle increasing data volume and user queries without sacrificing performance, making it a scalable and production-ready solution.

Model Implementation and Testing

Transaction:

Introduction:

In modern e-commerce systems, maintaining data consistency across various modules such as inventory, orders, payments, and customer data is critical. MongoDB, while schema-flexible, provides support for **multi-document transactions** that enable ACID-compliant operations across multiple collections. This is especially important in cases involving financial transactions, stock updates, and customer interactions where atomicity is essential. To strengthen data integrity and system reliability, the following transactions can be implemented in our project:

- **Return Order Transaction** – Handles returns by refunding payment and restoring inventory.
- **Customer Account Deletion Transaction** – Ensures data privacy by cleaning all user-linked records safely.
- **Failed Payment Rollback Transaction** – Prevents order creation and stock deduction if payment fails.
- **Update Inventory Transaction** – Safely adjusts product stock while logging changes.
- **Apply Discount/Promotion Transaction** – Validates and applies promotional codes while locking discount data.
- **Place Order Transaction** – Ensures atomic creation of an order, inventory deduction, cart clearance, and payment recording.
- **Cancel Order Transaction** – Ensures consistency when cancelling an order by refunding payment and restoring product stock

Implementing these transactions will ensure that all critical business workflows maintain strong consistency and fault tolerance. Our current implementation includes two core transactions that are to place order and cancel order transactions.

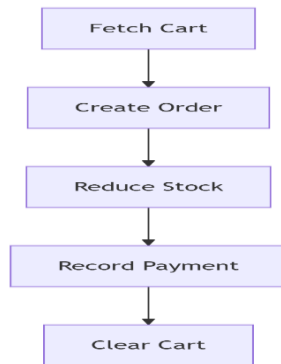
Connect to MongoDB Atlas:

```
PS E:\ADB final\Python code> & "C:/Program Files/Python313/python.exe" "e:/ADB final/Python code/connect_mongo.py"
Is mongos: False
Replica set name: atlas-3li90l-shard-0
PS E:\ADB final\Python code> |
```

This shows that MongoDB Atlas cluster is configured as a replica set, as intended and the connection works correctly, and client can communicate with it.

Transaction 1: Place Order Transaction

i. Flow-chart of Steps:



This ensures the customer isn't charged or sent an order if the stock isn't available or another operation fails.

ii. Before Running the Transaction:

Cart:

```
_id: "Cart1"
customer_id: "Customer1"
▼ items: Array (1)
  ▼ 0: Object
    product_id: "Product1"
    quantity: 2
    total_price: 3000
```

Product Stock:

```
_id: "Product1"
name : "Samsung Galaxy S23"
description : "Flagship smartphone with 256GB storage and 8GB RAM."
price : 250000
stock : 28
category_id : "Category1"
► attributes : Object
supplier_id : "Supplier1"
is_deleted : false
```

iii. Transaction Outcome:

```
PS E:\ADB final\Python code> & "C:/Program Files/Python313/python.exe" "e:/ADB final/Python code/Transaction1.py"
Order created: 68374091877722ad65a6b6ae
✔ Transaction completed. Order: 68374091877722ad65a6b6ae
Success! Order ID: 68374091877722ad65a6b6ae
PS E:\ADB final\Python code>
```

Order:

```
_id: ObjectId('68374091877722ad65a6b6ae')
customer_id : "Customer1"
▼ items : Array (1)
  ▼ 0: Object
    product_id : "Product1"
    quantity : 2
total_price : 3000
status : "Pending"
order_date : 2025-05-28T16:57:53.552+00:00
```

Product:

```
_id: "Product1"
name : "Samsung Galaxy S23"
description : "Flagship smartphone with 256GB storage and 8GB RAM."
price : 250000
stock : 26
category_id : "Category1"
► attributes : Object
supplier_id : "Supplier1"
is_deleted : false
```

Payment:

```
_id: ObjectId('68374091877722ad65a6b6af')
order_id : ObjectId('68374091877722ad65a6b6ae')
customer_id : "Customer1"
amount : 3000
payment_method : "Credit Card"
status : "Paid"
transaction_date : 2025-05-28T16:57:53.939+00:00
```

Cart:

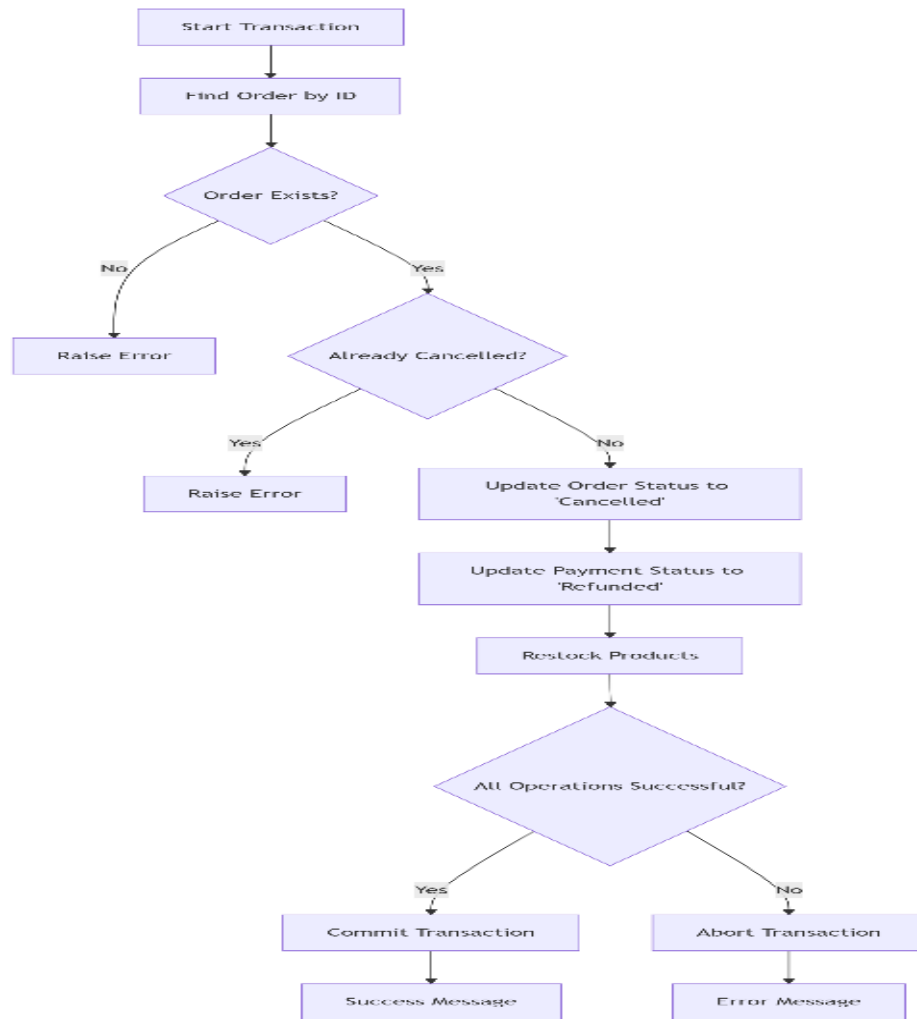
```
_id: "Cart1"
customer_id: "Customer1"
▶ items: Array (empty)
total_price: 0
```

iv. Summary of Transaction on collections:

Collection	Operation	Reason	Key Fields
Orders	insert_one	Create immutable order record	items, total_price, status: "Pending"
Payments	insert_one	Record payment transaction	order_id, amount, status: "Paid"
Products	update_one	Reduce inventory stock	stock: -quantity (atomic decrement)
`Cart	update_one	Clear user's cart after checkout	items: [], total_price: 0

Transaction 2: Cancel Order Transaction

i. Flow Chart of steps:



This transaction ensures to avoid inconsistent states like restocking without refunding or refunding without changing order status.

ii. Before Running the Transaction:

Products:

```

_id: "Product1"
name: "Samsung Galaxy S23"
description: "Flagship smartphone with 256GB storage and 8GB RAM."
price: 250000
stock: 26
category_id: "Category1"
attributes: Object
supplier_id: "Supplier1"
is_deleted: false
  
```

Orders:

```
_id: ObjectId('68373b6110be9a56c3035a5b')
customer_id: "Customer1"
▼ items: Array (1)
  ▼ 0: Object
    product_id: "Product1"
    quantity: 2
    price: 250000
total_price: 500000
status: "Pending"
order_date: 2025-05-28T16:35:45.946+00:00
```

Payments:

```
_id: ObjectId('68373b6210be9a56c3035a5c')
order_id: ObjectId('68373b6110be9a56c3035a5b')
customer_id: "Customer1"
amount: 500000
payment_method: "Credit Card"
status: "Paid"
transaction_date: 2025-05-28T16:35:46.290+00:00
```

iii. Transaction Outcome:

```
PS E:\ADB final\Python code> & "C:/Program Files/Python313/python.exe" "e:/ADB final/Python code/Order_Cancellation.py"
Transaction started successfully
✔ Order 68373b6110be9a56c3035a5b cancelled successfully
PS E:\ADB final\Python code> █
```

Products:

```
_id: "Product1"
name: "Samsung Galaxy S23"
description: "Flagship smartphone with 256GB storage and 8GB RAM."
price: 250000
stock: 28
category_id: "Category1"
► attributes: Object
  supplier_id: "Supplier1"
  is_deleted: false
```

Payments:

```
_id: ObjectId('68373b6210be9a56c3035a5c')
order_id: ObjectId('68373b6110be9a56c3035a5b')
customer_id: "Customer1"
amount: 500000
payment_method: "Credit Card"
status: "Refunded"
transaction_date: 2025-05-28T16:35:46.290+00:00
refund_date: 2025-05-28T17:54:16.221+00:00
```

Orders:

```
_id: ObjectId('68373b6110be9a56c3035a5b')
customer_id: "Customer1"
items: Array (1)
total_price: 500000
status: "Cancelled"
order_date: 2025-05-28T16:35:45.946+00:00
cancelled_at: 2025-05-28T17:54:16.038+00:00
```

iv. Summary of Transaction on collections:

Collection	Operation	Reason	Field Changes
Orders	update_one	Mark order as cancelled (immutable status change)	status: "Cancelled", cancelled_at: timestamp
Payments	update_one	Update payment status to refunded	status: "Refunded", refund_date: timestamp
Products	update_one	Restock inventory (atomic increment)	stock: +quantity (for each product)

➤ Conclusion:

Transactional operations in MongoDB are vital for building robust, consistent, and reliable e-commerce platforms. By implementing Place Order and Cancel Order transactions, we've established a foundational layer of integrity for critical workflows. However, to fully cover all potential edge cases and maintain seamless customer experience, it is essential to expand this coverage to include returns, account deletion, payment rollbacks, inventory updates, and discount applications.

Integrating these transactions will not only protect against data anomalies but also enhance trust, compliance, and operational accuracy across the system.

Concurrency Control

Identify Collections for Locking

1. Cart (Optimistic)

Used for temporary user data with frequent but low-conflict updates. Optimistic locking allows non-blocking edits and better performance, especially in shared or collaborative cart scenarios.

2. Products (Optimistic + Pessimistic for Inventory)

Product details (like name or price) are updated infrequently, so optimistic locking works

well. However, inventory levels require strict control to prevent overselling, so pessimistic locking is used during stock reservation.

3. Orders (Pessimistic)

Order processing involves multiple critical operations (payment, inventory, shipping). Pessimistic locking ensures atomicity and prevents race conditions or duplicate orders during checkout.

4. Payments (Pessimistic)

Payments are sensitive financial operations that must be processed exactly once. Pessimistic locking guarantees consistency, avoids double charges, and maintains compliance with financial regulations.

➤ Why

- Cart and Products face high concurrent updates.
- Orders/Payments need atomicity for financial integrity.

Optimistic Locking

Optimistic locking ensures data consistency in concurrent environments by allowing multiple users to read records simultaneously but detecting conflicts during updates. In this project, it safeguards cart modifications and inventory updates by validating document versions before applying changes. This approach minimizes database locks while preventing overselling or duplicate transactions, ideal for high-traffic e-commerce systems. In this we add a version field to detect conflicts during concurrent updates and with each successful update increases the version to ensure data consistency.

Queries that add version field in collections:

```
> db.Cart.updateMany({}, { $set: { version: 1 } })
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 16,
  modifiedCount: 16,
  upsertedCount: 0
}
```

```
> db.Products.updateMany({}, { $set: { version: 1 } })
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 20,
  modifiedCount: 20,
  upsertedCount: 0
}
```

Optimistic Locking in Cart Updates:

- Update Item Quantity and Total Price in Cart:

i. Before:

```
{
  "_id": "Cart20",
  "customer_id": "Customer120",
  "items": [
    {
      "product_id": "Product30",
      "quantity": 1
    },
    {
      "product_id": "Product5",
      "quantity": 1
    }
  ],
  "total_price": 210000,
  "version": 1
}
```

ii. Python Code implementation in Terminal:

```
PS E:\ADB final\Python code> & "C:/Program Files/Python313/python.exe" "e:/ADB final/Python code/CartService.py"
=== STEP 1: Connection Established ===
Database: EcommerceInventoryManagment, collection: Cart
=====

=== STEP 2: Checking Cart Existence ===
Looking for cart with _id: Cart20
✔ Cart found:
{'_id': 'Cart20', 'customer_id': 'Customer120', 'items': [{'product_id': 'Product30', 'quantity': 3}, {'product_id': 'Product5', 'quantity': 1}], 'total_price': 0, 'version': 2}
=====

🔧 Starting change_quantity operation
📋 Cart: Cart20, Product: Product30, New Qty: 3

=== STEP 2: Checking Cart Existence ===
Looking for cart with _id: Cart20
✔ Cart found:
{'_id': 'Cart20', 'customer_id': 'Customer120', 'items': [{'product_id': 'Product30', 'quantity': 3}, {'product_id': 'Product5', 'quantity': 1}], 'total_price': 0, 'version': 2}
✖ Found product at index 0
🔄 Updating total from 0 to 0

=== STEP 3: Attempting Update ===
🔒 Lock check: Version must be 2
📝 Proposed changes: {'items.0.quantity': 3, 'total_price': 0}
✔ Update successful! New version: 3
=====

🟢 Final result: SUCCESS
=====
PS E:\ADB final\Python code>
```

iii. After:

```
_id: "Cart20"
customer_id: "Customer120"
▼ items: Array (2)
  ▼ 0: Object
    product_id: "Product30"
    quantity: 3
  ▼ 1: Object
    product_id: "Product5"
    quantity: 1
total_price: 212000
version: 3
```

iv. Cart20 Updates:

- ITEM: Product30 quantity changed (1 → 3)
- PRICE: Total updated (210000 → 212000)
- VERSION: Incremented (2 → 3)

Optimistic locking in product updates:

- Update Stock and Price of Product:

i. Before:

```
▼ {
  "_id": "Product19",
  "name": "Lenovo ThinkPad X1 Carbon",
  "description": "14-inch business laptop with Intel Core i7 and 32GB RAM.",
  "price": 350000,
  "stock": 7,
  "category_id": "Category9",
  ▼ "attributes": {
    "color": "Black",
    "size": "14-inch"
  },
  "supplier_id": "Supplier1",
  "is_deleted": false,
  "version": 1
}
```

ii. After:

```
_id: "Product19"
name: "Lenovo ThinkPad X1 Carbon"
description: "14-inch business laptop with Intel Core i7 and 32GB RAM."
price: 370000
stock: 2
category_id: "Category9"
► attributes: Object
supplier_id: "Supplier1"
is_deleted: false
version: 2
```

iii. Product19 Updates:

- **Stock Reduction:** STOCK: 7 → 2
- **Price Update :** PRICE: 350000 → 370000
- **Version:** 1→2

Pessimistic Locking:

Pessimistic locking ensures exclusive access to a database document during critical operations, preventing concurrent modifications. It works by "locking" the document for a specific process until the operation completes or a timeout occurs. This approach is ideal for financial transactions or inventory management where data consistency is crucial. Unlike optimistic locking, it proactively blocks other processes rather than detecting conflicts after they occur.

Quires that add fields for locking in Orders and Payment collections

```
> db.Orders.updateMany({}, {
  $set: {
    locked: false,
    lock_owner: null,
    lock_expiry: null
  }
})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 17,
  modifiedCount: 17,
  upsertedCount: 0
}
```

```
> db.Payments.updateMany({}, {
  $set: {
    locked: false,
    lock_owner: null,
    lock_expiry: null
  }
})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 17,
  modifiedCount: 17,
  upsertedCount: 0
}
```

i. Before:

The document is in its initial state with no active locks. All locking fields (locked, lock_owner, lock_expiry) are empty or set to default values.

```
▼ {
  "_id": "Payment7",
  "order_id": "Order8",
  "customer_id": "Customer8",
  "amount": 2600,
  "payment_method": "Debit Card",
  "status": "Pending",
  ▼ "transaction_date": {
    "$date": "2025-03-02T00:00:00.000Z"
  },
  "lock_expiry": null,
  "lock_owner": null,
  "locked": false
}
```

```
▼ {
  "_id": "Order8",
  "customer_id": "Customer8",
  ▼ "items": [
    ▼ {
      "product_id": "Product11",
      "quantity": 4,
      "price": 650
    }
  ],
  "total_price": 2600,
  "status": "Pending",
  ▼ "order_date": {
    "$date": "2025-03-01T00:00:00.000Z"
  },
  "lock_expiry": null,
  "lock_owner": null,
  "locked": false
}
```

ii. Acquired Lock:

The document is now locked, showing locked: true, the owner's ID, and an expiry timestamp. No other process can modify it until the lock is released or expires.

```
_id: "Order8"
customer_id: "Customer8"
▶ items: Array (1)
total_price: 2600
status: "Pending"
order_date: 2025-03-01T00:00:00.000+00:00
lock_expiry: 2025-05-28T22:34:44.815+00:00
lock_owner: "Customer8"
locked: true
```

The lock remains active while the payment is processed. The status is still "Pending," indicating the transaction is ongoing.

```
_id: "Payment7"
order_id: "Order8"
customer_id: "Customer8"
amount: 2600
payment_method: "Debit Card"
status: "Pending"
transaction_date: 2025-03-02T00:00:00.000+00:00
lock_expiry: 2025-05-28T22:49:59.711+00:00
lock_owner: "Customer8"
locked: true
```

iii. After:

The lock fields are reset, and the status updates to "Completed" and "Processed". The document is now available for other operations.

```
_id: "Payment7"
order_id: "Order8"
customer_id: "Customer8"
amount: 2600
payment_method: "Debit Card"
status: "Completed"
transaction_date: 2025-03-02T00:00:00.000+00:00
lock_expiry: null
lock_owner: null
locked: false
```

```
_id: "Order8"
customer_id: "Customer8"
items: Array (1)
total_price: 2600
status: "Processed"
order_date: 2025-03-01T00:00:00.000+00:00
lock_expiry: null
lock_owner: null
locked: false
```

Pessimistic locking provides strong consistency for critical operations by ensuring exclusive access to data. While it introduces some overhead due to blocking, it prevents race conditions in high-stakes scenarios like payments or inventory updates. Proper timeout handling and atomic lock acquisition are essential to avoid deadlocks. This approach complements optimistic locking, which is better suited for high-concurrency, low-conflict systems.

Create Views in MongoDB Shell

1. View: Product Summary

```
db.createView(
```

```
"ProductSummary",  
"Products",  
[{ $project: { name: 1, price: 1, category_id: 1, stock: 1, _id: 0 } } ]  
)
```

```
> _MONGOSH  
  
> use EcommerceInventoryManagment  
< switched to db EcommerceInventoryManagment  
> db.createView(  
  "ProductSummary",  
  "Products",  
  [  
    { $project: { name: 1, price: 1, category_id: 1, stock: 1, _id: 0 } }  
  ]  
)  
< { ok: 1 }  
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>
```

2. View: Order Summary (hide sensitive data)

```
db.createView(  
  "OrderSummary",  
  "Orders",  
  [  
    { $project: { customer_id: 1, total_price: 1, status: 1, _id: 0 } }  
  ]  
)
```

```
> db.createView(  
  "OrderSummary",  
  "Orders",  
  [  
    { $project: { customer_id: 1, total_price: 1, status: 1, _id: 0 } }  
  ]  
)  
< { ok: 1 }  
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>
```

3. View : Customers View (hide phone, show address)

```
db.createView(  
  "CustomerPublicView",  
  "Customers",  
  [  
    {  
      $project: {  
        name: 1,  
        email: 1,  
        "address.city": 1,  
        "address.country": 1,  
        _id: 0  
      }  
    }  
  ]  
)
```

```
> db.createView(  
  "CustomerPublicView",  
  "Customers",  
  [  
    {  
      $project: {  
        name: 1,  
        email: 1,  
        "address.city": 1,  
        "address.country": 1,  
        _id: 0  
      }  
    }  
  ]  
)  
< { ok: 1 }  
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment> |
```

4. View: Payments View (show paid only)

```
db.createView(  
  "SuccessfulPaymentsView",  
  "Payments",  
  [  
    { $match: { status: "Paid" } }, { $project: {  
      order_id: 1,  
      customer_id: 1,  
      amount: 1,  
      payment_method: 1,  
      transaction_date: 1,  
      _id: 0 }  
    }  
  ]  
)
```

```

> db.createView(
  "SuccessfulPaymentsView",
  "Payments",
  [
    { $match: { status: "Paid" } },
    {
      $project: {
        order_id: 1,
        customer_id: 1,
        amount: 1,
        payment_method: 1,
        transaction_date: 1,
        _id: 0
      }
    }
  ]
)
< { ok: 1 }
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>

```

5. View: Reviews View (high rating only)

```

db.createView(
  "TopRatedReviewsView",
  "Reviews",
  [{ $match: { rating: { $gte: 4 } } },
  {$project: {
    product_id: 1,
    customer_id: 1,
    rating: 1,
    review_text: 1,
    review_date: 1,
    _id: 0}}]
)

```

```

> db.createView(
  "TopRatedReviewsView",
  "Reviews",
  [
    { $match: { rating: { $gte: 4 } } },
    {
      $project: {
        product_id: 1,
        customer_id: 1,
        rating: 1,
        review_text: 1,
        review_date: 1,
        _id: 0
      }
    }
  ]
)
< { ok: 1 }
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>

```

6. **View: Suppliers View (summary only) as "SupplierInfoView"**
7. **View: Shipping View (in-transit only) as "ActiveShipmentsView"**
8. **View: Carts View (show only cart totals) as "CartSummaryView"**

Successful query after creating views for Reviews View (high rating only)

```
db.TopRatedReviewsView.find()
```

```
>_MONGOSH
> use EcommerceInventoryManagment
< switched to db EcommerceInventoryManagment
> db.TopRatedReviewsView.find()
< [
  {
    customer_id: 'Customer1',
    product_id: 'Product1',
    rating: 5,
    review_text: 'Amazing phone with great battery life!',
    review_date: 2025-03-23T00:00:00.000Z
  },
  {
    customer_id: 'Customer2',
    product_id: 'Product2',
    rating: 4,
    review_text: 'Great laptop but a bit expensive.',
    review_date: 2025-03-22T00:00:00.000Z
  },
  {
    customer_id: 'Customer3',
    product_id: 'Product3',
    rating: 5,
    review_text: 'Super comfortable shoes!',
    review_date: 2025-03-21T00:00:00.000Z
  }
]
```

Materialized View

1. View: Monthly Sales Summary

```
db.Orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: { $substr: ["$order_date", 0, 7] }, // YYYY-MM
      total_sales: { $sum: "$total_price" },
      order_count: { $sum: 1 }
    }
  },
  { $out: "MonthlySalesSummary" } // stores permanently
])
```

```

>_MONGOSH
<
> db.Orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: { $substr: ["$order_date", 0, 7] }, // YYYY-MM
      total_sales: { $sum: "$total_price" },
      order_count: { $sum: 1 }
    }
  },
  { $out: "MonthlySalesSummary" } // stores permanently
])
<
> db.MonthlySalesSummary.find().pretty()
< {
  _id: '2025-02',
  total_sales: 31600,
  order_count: 8
}
{
  _id: '1970-01',
  total_sales: 0,
  order_count: 1
}
{

```

2. View: Top-Spending Customers (from Orders)

```

db.Orders.aggregate([
  $group: {
    _id: "$customer_id",
    totalSpent: { $sum: "$total_price" },
    orderCount: { $sum: 1 }},
  { $sort: { totalSpent: -1 } },
  { $out: "TopSpenders" }])

```

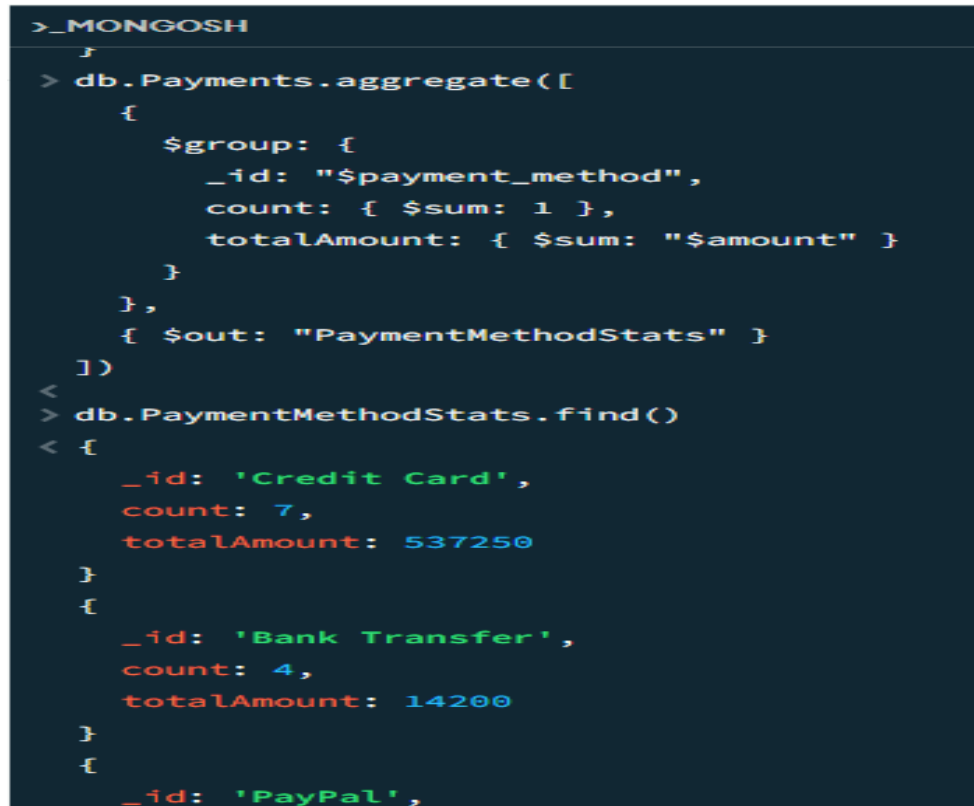
```

>_MONGOSH
> db.Orders.aggregate([
  $group: {
    _id: "$customer_id",
    totalSpent: { $sum: "$total_price" },
    orderCount: { $sum: 1 }
  },
  { $sort: { totalSpent: -1 } },
  { $out: "TopSpenders" }
])
<
> db.TopSpenders.find().sort({ totalSpent: -1 })
< {
  _id: 'Customer1',
  totalSpent: 503000,
  orderCount: 3
}
{
  _id: 'Customer7',
  totalSpent: 9000,
  orderCount: 1
}
{
  _id: 'Customer3',
  totalSpent: 5400,
  orderCount: 1
}

```

3. View: Frequent Payment Methods (from Payments)

```
db.Payments.aggregate([{\n  $group: {\n    _id: "$payment_method",\n    count: { $sum: 1 },\n    totalAmount: { $sum: "$amount" }},\n  {\n    $out: "PaymentMethodStats" }])
```



```
> _MONGOSH\n>\n> db.Payments.aggregate([\n  {\n    $group: {\n      _id: "$payment_method",\n      count: { $sum: 1 },\n      totalAmount: { $sum: "$amount" }\n    }\n  },\n  {\n    $out: "PaymentMethodStats" }\n])\n<\n> db.PaymentMethodStats.find()\n< {\n  _id: 'Credit Card',\n  count: 7,\n  totalAmount: 537250\n}\n{\n  _id: 'Bank Transfer',\n  count: 4,\n  totalAmount: 14200\n}\n{\n  _id: 'PayPal',
```

4. View: Most Reviewed Products (from Reviews) as "ProductReviewStats"
5. View: Shipping Delay Report (from Shipping) as "ShippingDelayReport"
6. View: Inventory Summary by Category (from Products) as "CategoryInventorySummary"

Simulate Access Control (No User Management in Current Version)

We couldn't use `db.createUser()` without enabling **authentication mode** manually.

Instead, simulate role-based separation with **different views** which we have already created successfully in the database.

You give access to users via **their application** or a custom UI not the DB.

NoSQL Injection

1. Identify Injection Risk Points

Collections at risk:

- Customers — login forms (email/password)
- Orders — user filters/search
- Products — category or price filters
- Payments — status/date filters

Dangerous operations:

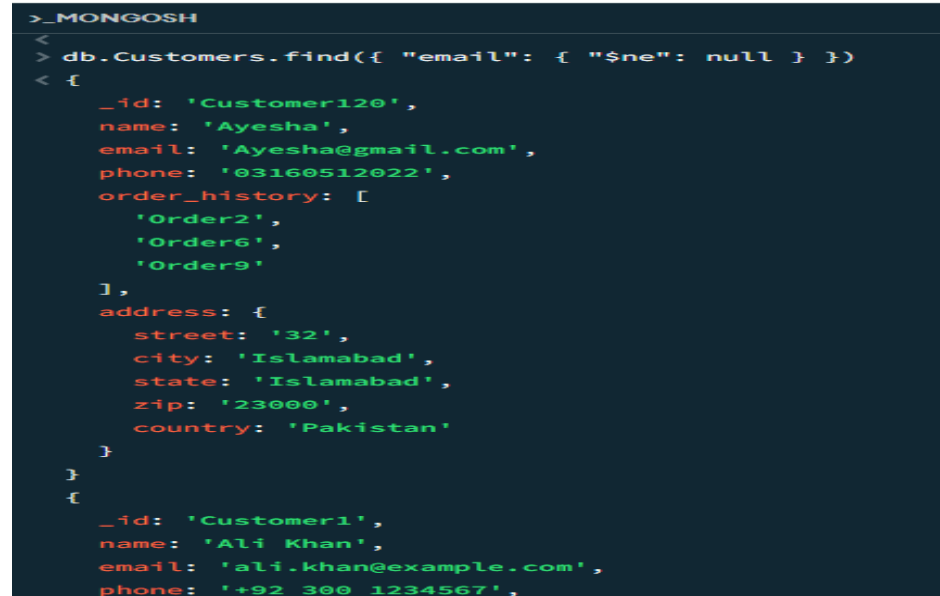
- Passing **raw user input directly into queries**
- Using \$where, \$function, or **non-sanitized JSON**

2. Test the Attack (to Understand the Risk)

Simulated an injection attack on Customers collection.

```
db.Customers.find({ "email": { "$ne": null } })
```

It returned **all customers**, it means the filter is vulnerable.



```
>_MONGOSH
<
> db.Customers.find({ "email": { "$ne": null } })
< {
  _id: 'Customer120',
  name: 'Ayesha',
  email: 'Ayesha@gmail.com',
  phone: '03160512022',
  order_history: [
    'Order2',
    'Order6',
    'Order9'
  ],
  address: {
    street: '32',
    city: 'Islamabad',
    state: 'Islamabad',
    zip: '23000',
    country: 'Pakistan'
  }
}
{
  _id: 'Customer1',
  name: 'Ali Khan',
  email: 'ali.khan@example.com',
  phone: '+92 300 1234567',
```

3. Replace with Strict Filtered Query

```
var email = "user@example.com";
if (/^[w\.-]+@[w\.-]+\w{2,4}$/.test(email)) {
  db.Customers.find({ "email": email });} else {
  print("Invalid email format. Possible injection attempt.");}
```

This prevents \$ne, \$gt, \$regex injections from being passed.

The Real Risk is if we would have been using a **web interface or API** that accepts user input (e.g. email: { \$ne: null }), then attackers can pass the query

```
db.Customers.find({ "email": { "$ne": null } })
```

and they can Bypass login, Expose multiple accounts, Exfiltrate data

But in the **MongoDB Shell**, we are running the test directly — so it's our own query. No vulnerability there.

4. Ban the Use of \$where, \$function (Dangerous)

Why ? because it lets attackers inject arbitrary JavaScript.

So we used proper operators:

```
db.Products.find({
  stock: { $lt: 5 }
})
```

```
>_MONGOSH
}
>
db.Products.find({
  $where: "this.stock < 5"
})
❌ ▶ MongoServerError[AtlasError]: $where not allowed in this atlas tier
> db.Products.find({
  stock: { $lt: 5 }
})
< {
  _id: 'Product7',
  name: 'Rolex Submariner',
  description: 'Luxury automatic wristwatch.',
  price: 2500000,
  stock: 3,
  category_id: 'Category7',
  attributes: {
    color: 'Gold',
    size: '40mm'
  },
  supplier_id: 'Supplier7',
  is_deleted: false
}
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>
```

5. Protect Admin Filters (Querying with Inputs)

We were writing the query with inputs,so we **manually validated allowed fields**.

```
var allowedFields = ["name", "category_id", "price"];
var field = "price";
var value = 1000;
if (allowedFields.includes(field) && typeof value === "number") {var query = {};query[field] = { $lt: value };
  db.Products.find(query);}
else {print("Invalid query structure");
}
```

```
> var allowedFields = ["name", "category_id", "price"];
var field = "price";
var value = 1000;

if (allowedFields.includes(field) && typeof value === "number") {
  var query = {};
  query[field] = { $lt: value }; // safe operation
  db.Products.find(query);
} else {
  print("Invalid query structure");
}
<
Atlas atlas-3li90l-shard-0 [primary] EcommerceInventoryManagment>
```

6. Disable JavaScript Operators in Queries

In our queries,we just **avoided using** JavaScript Operators. All queries used regular operators like \$match, \$lt, \$gt, \$eq, \$regex (only if needed).

Because JavaScript Operators allow **arbitrary code execution** inside the database.

7. Final Secure Query Examples

i. Find orders by customer

```
db.Orders.find({ customer_id: "Customer12" })
```

```
> db.Orders.find({ customer_id: "Customer12" })
< {
  _id: 'Order12',
  customer_id: 'Customer12',
  items: [
    {
      product_id: 'Product17',
      quantity: 3,
      price: 1100
    }
  ],
  total_price: 3300,
  status: 'Delivered',
  order_date: 2025-02-24T00:00:00.000Z
}
```

ii. Products under price limit

```
db.Products.find({ price: { $lte: 100000 } })
```

```
>_MONGOSH
>
> db.Products.find({ price: { $lte: 100000 } })
< {
  _id: 'Product3',
  name: 'Nike Air Max 2023',
  description: 'Comfortable running shoes for men.',
  price: 22000,
  stock: 30,
  category_id: 'Category3',
  attributes: {
    color: 'White',
    size: '42'
  },
  supplier_id: 'Supplier3',
  is_deleted: false
}
{
  _id: 'Product11',
  name: 'Gucci Leather Wallet',
  description: 'Luxury leather wallet for men.',
  price: 45000,
  stock: 40,
  category_id: 'Category11',
  attributes: {
```

iii. Payments on date

```
db.Payments.find({ transaction_date: ISODate ("2025-03-02")})
```

```
>_MONGOSH
>
> db.Payments.find({ transaction_date: ISODate ("2025-03-02")})
< {
  _id: 'Payment1',
  order_id: 'Order1',
  customer_id: 'Customer1',
  amount: 5000,
  payment_method: 'Credit Card',
  status: 'Paid',
  transaction_date: 2025-03-02T00:00:00.000Z
}
{
  _id: 'Payment7',
  order_id: 'Order8',
  customer_id: 'Customer8',
  amount: 2600,
  payment_method: 'Debit Card',
  status: 'Pending',
  transaction_date: 2025-03-02T00:00:00.000Z
}
{
  _id: 'Payment14',
  order_id: 'Order15',
  customer_id: 'Customer15',
  amount: 4500,
```

iv. Find by exact match only

```
db.Customers.find({ email: "bilal.hussain@example.com"})
```

```
> db.Customers.find({ email: "bilal.hussain@example.com"})
< {
  _id: 'Customer5',
  name: 'Bilal Hussain',
  email: 'bilal.hussain@example.com',
  phone: '+92 310 5678901',
  order_history: [
    'Order5',
    'Order6',
    'Order7'
  ],
  address: {
    street: 'Jinnah Avenue',
    city: 'Islamabad',
    state: 'Capital Territory',
    zip: '44000',
    country: 'Pakistan'
  }
}
```

Atlas atlas-3l190l-shard-0 [primary] EcommerceInventoryManagment>

User Manual for Graphic User interface:

Project Setup

To run the GUI-based project, the following external libraries must be added to the project's ClassPath:

- mongodb-driver-sync-5.5.0.jar
- bson-5.5.0.jar
- mongodb-driver-core-5.5.0.jar
- jfreechart-1.0.19.jar
- jcommon-1.0.23.jar

Once these libraries are added, the project will compile and run without any dependency issues.

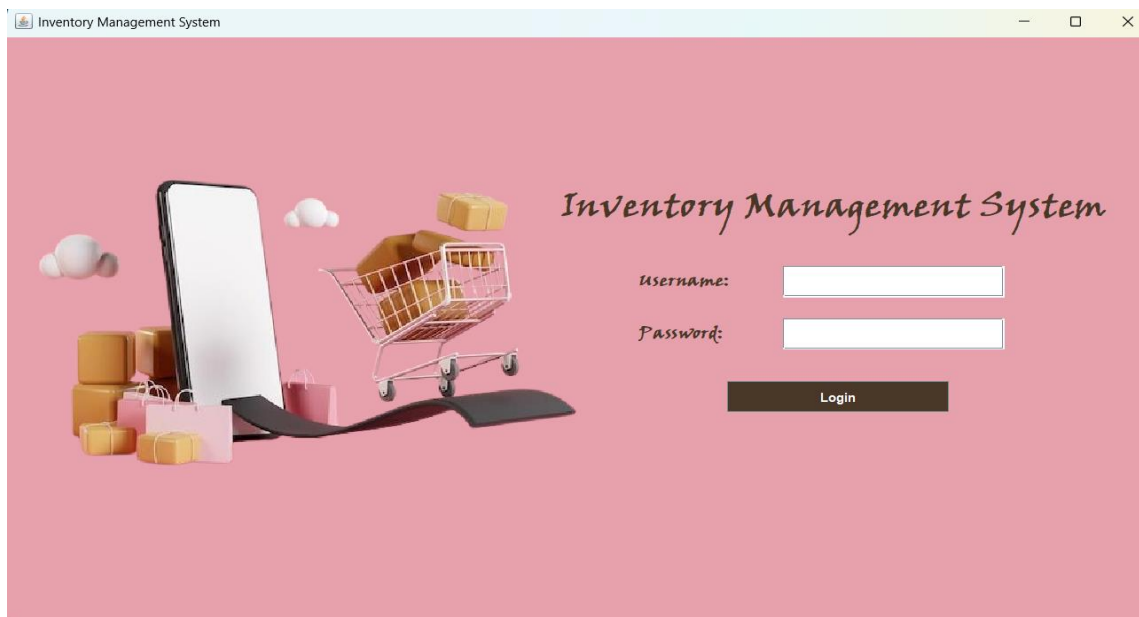
Login Window:

Upon launching the application, the **Login Window** is the first interface presented. It requires the user to input valid credentials to access the database. This ensures that only authorized users can proceed further.

Authorized Users:

Username	Password
Urva	101
Rameesha	087
Ayesha	128

If incorrect credentials are entered, an error message is displayed, denying access.



Menu Window:

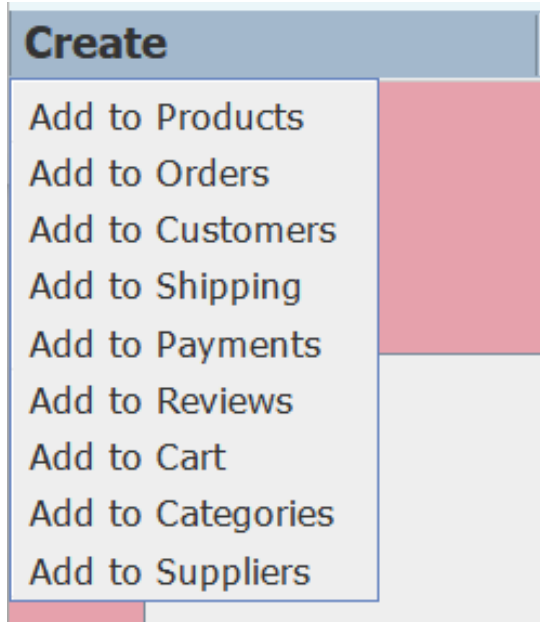
Upon successful login, the user is redirected to the **Menu Window**, which provides navigation for all CRUD operations and aggregation queries. It features:

- Menus for **Create, Read, Update, Delete**
- A **Queries** menu for predefined aggregation queries
- A **table** to display query results
- Buttons for **Summary Statistics** and **Chart Visualizations**

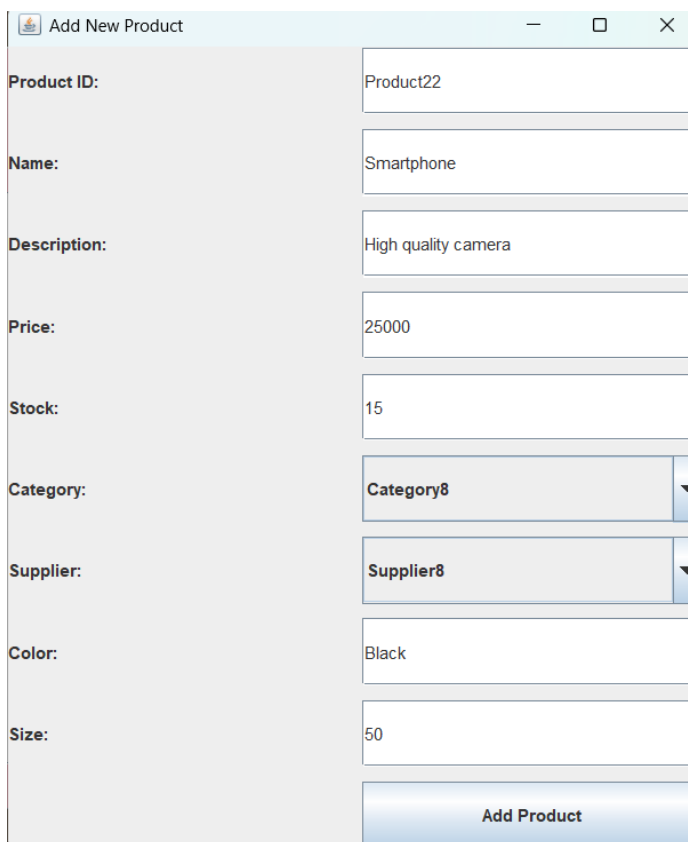


Create Menu:

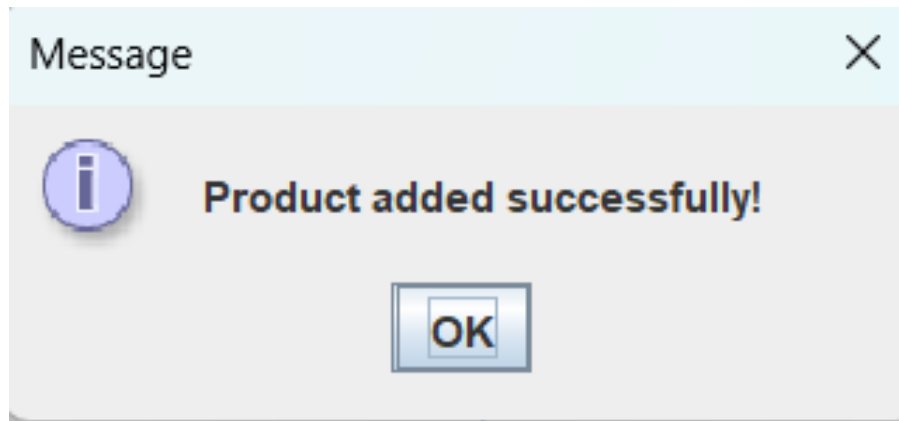
The **Create** menu allows users to add new documents to any of the database collections. Selecting a submenu under Create launches a corresponding input form.



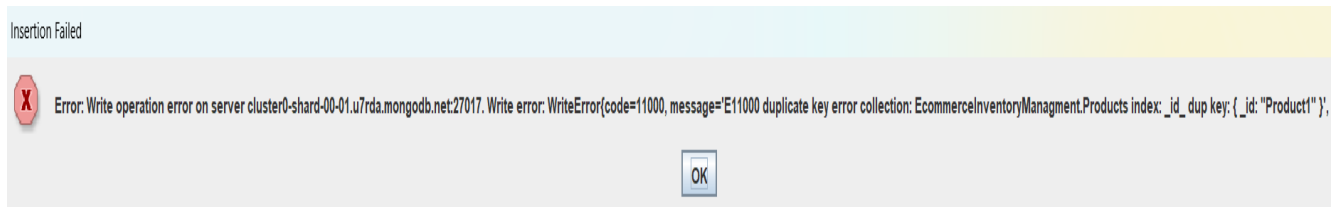
- Each form collects the necessary data to create a document.

A screenshot of a web application window titled 'Add New Product'. The window contains a form with the following fields: 'Product ID:' with value 'Product22', 'Name:' with value 'Smartphone', 'Description:' with value 'High quality camera', 'Price:' with value '25000', 'Stock:' with value '15', 'Category:' with a dropdown menu showing 'Category8', 'Supplier:' with a dropdown menu showing 'Supplier8', 'Color:' with value 'Black', and 'Size:' with value '50'. At the bottom of the form is a blue button labeled 'Add Product'.

- If all inputs are valid, the data is inserted into the database.



- For collections like **Orders**, ensure that a unique Order ID is provided to prevent duplication errors.



All the other forms work similarly.

Read Menu:

The **Read** menu retrieves and displays documents from the collections in a tabular format. Each submenu corresponds to a collection, and selecting one displays its data in an easy-to-read table.



Here is an example of data display:



The screenshot shows a web application titled "Inventory Management System" with a navigation bar containing "Create", "Read", "Update", "Delete", and "Queries". The main content area has a pink background with the title "Inventory Management System" in a cursive font. Below the title is a table with 6 columns: _id, customer_id, product_id, rating, review_text, and review_date. The table contains 21 rows of review data. At the bottom of the interface are two buttons: "Show Summary Statistics" and "Display Charts".

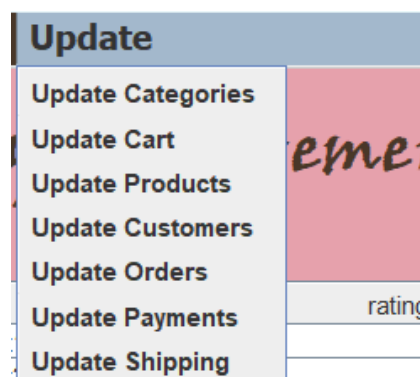
_id	customer_id	product_id	rating	review_text	review_date
Review1	Customer1	Product1	5	Amazing phone with great b	Sun Mar 23 05:00:00 PKT ...
Review2	Customer2	Product2	4	Great laptop but a bit expen	Sat Mar 22 05:00:00 PKT 2.
Review3	Customer3	Product3	5	Super comfortable shoes!	Fri Mar 21 05:00:00 PKT 20.
Review4	Customer4	Product4	4	Cooling is good but installati	Thu Mar 20 05:00:00 PKT 2.
Review5	Customer5	Product5	5	Very spacious fridge, works	Wed Mar 19 05:00:00 PKT ...
Review6	Customer6	Product6	5	Best noise-canceling headp	Tue Mar 18 05:00:00 PKT 2.
Review7	Customer7	Product7	5	Luxury at its best!	Mon Mar 17 05:00:00 PKT ...
Review8	Customer8	Product8	4	Excellent display, but price i	Sun Mar 16 05:00:00 PKT ...
Review9	Customer9	Product9	5	M2 chip is blazing fast!	Sat Mar 15 05:00:00 PKT 2.
Review10	Customer10	Product10	4	Good camera but expensiv...	Fri Mar 14 05:00:00 PKT 20.
Review11	Customer11	Product11	5	Stylish and elegant!	Thu Mar 13 05:00:00 PKT 2.
Review12	Customer12	Product12	5	Toyota never disappoints!	Wed Mar 12 05:00:00 PKT ...
Review13	Customer13	Product13	5	Fantastic tablet for work	Tue Mar 11 05:00:00 PKT 2.
Review14	Customer14	Product14	4	Love the PS5, but stock iss...	Mon Mar 10 05:00:00 PKT ...
Review15	Customer15	Product15	5	Awesome TV with great pic...	Sun Mar 09 05:00:00 PKT ...
Review16	Customer16	Product16	5	Fastest graphics card ever!	Sat Mar 08 05:00:00 PKT 2.
Review17	Customer17	Product17	5	Crystal clear sound!	Fri Mar 07 05:00:00 PKT 20.
Review18	Customer18	Product18	4	Battery life could be better.	Thu Mar 06 05:00:00 PKT 2.
Review19	Customer19	Product19	5	Very durable laptop.	Wed Mar 05 05:00:00 PKT ...
Review20	Customer20	Product20	5	Drones work flawlessly!	Tue Mar 04 05:00:00 PKT 2.
Review21	Customer11	Product1	5	Excellent product	Sun May 25 18:04:05 PKT ...

All the other collections have similar display.

Update menu:

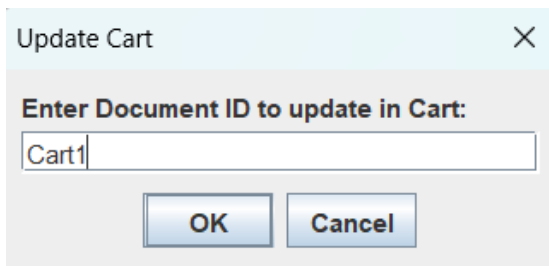
The **Update** menu allows modifications to specific fields in selected collections. The following operations are supported:

- **Categories:** Update description
- **Carts:** Add new items
- **Products:** Update description, price, and stock
- **Customers:** Add new orders in order history
- **Orders, Payments, Shippings:** Update status



Example - Adding Items to a Cart:

1. Enter the **Cart ID**.



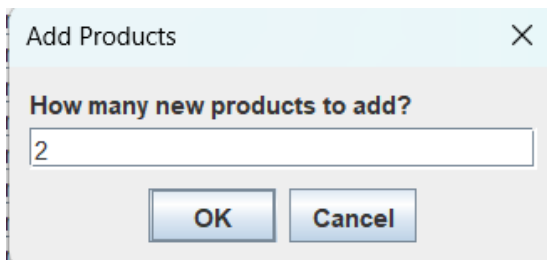
Update Cart

Enter Document ID to update in Cart:

Cart1

OK Cancel

2. Specify the number of items to add.



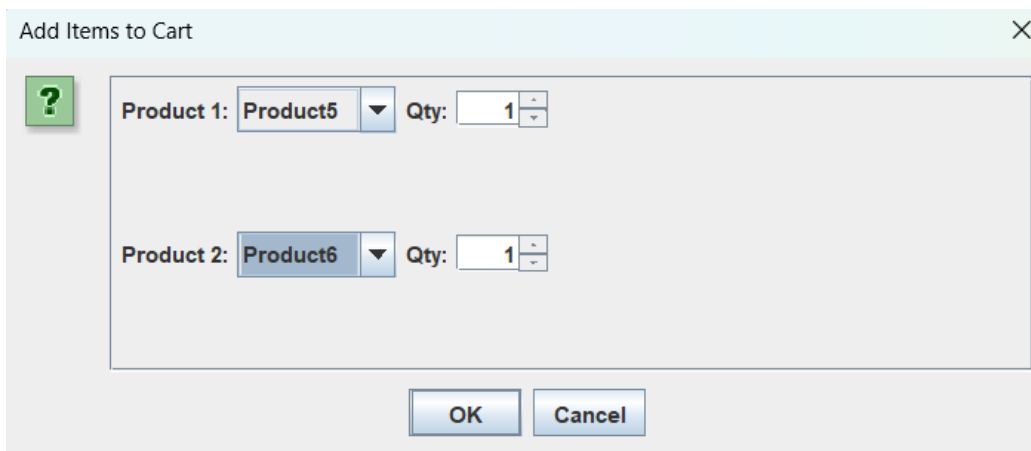
Add Products

How many new products to add?

2

OK Cancel

3. A form appears with a dropdown for selecting product IDs and entering quantities.



Add Items to Cart

?

Product 1: Product5 Qty: 1

Product 2: Product6 Qty: 1

OK Cancel

4. The Cart is updated upon submission.



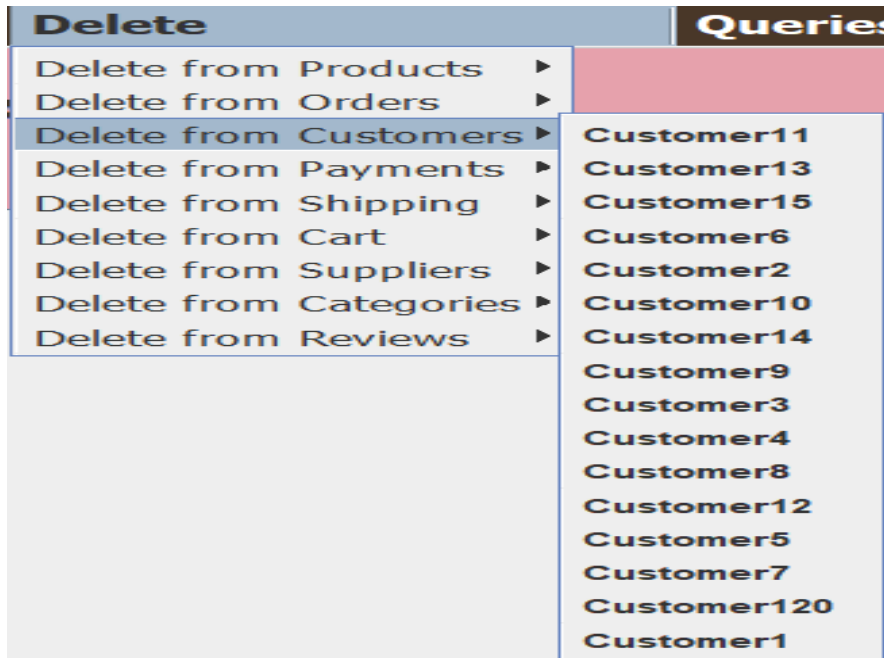
Success

Cart updated successfully.

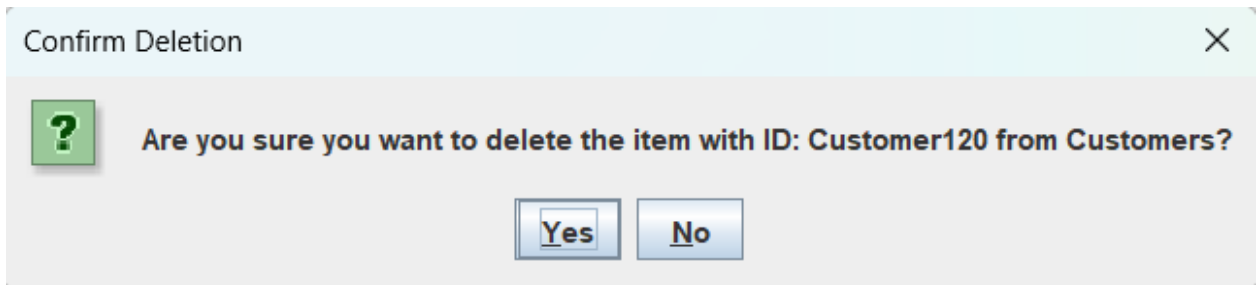
OK

Delete Menu:

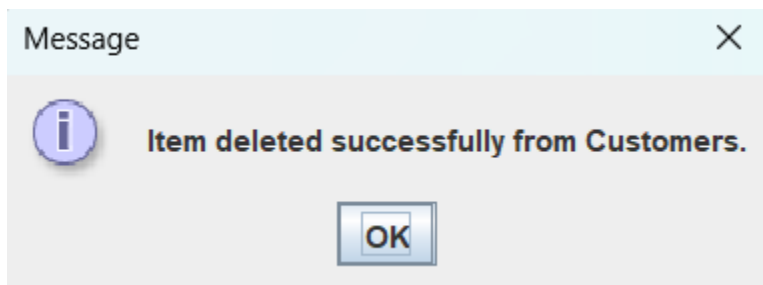
The **Delete** menu allows users to delete documents based on Object IDs. Each submenu provides a dropdown or list of existing Object IDs for selection.



- Users must confirm deletion before the action is executed.



- Once confirmed, the selected document is removed from the collection.



Queries menu:

The **Queries** menu includes a set of 11 predefined aggregation queries relevant to the project. These provide analytical insights into the database:

1. Top 5 Selling Products
2. High-Spending Customers
3. Most Popular Payment Method
4. Out-of-Stock Products
5. Customers with Most Orders
6. Customers with Most Reviews
7. Orders Not Delivered
8. Pending Payments
9. Products with Low Stock (threshold < 5)
10. Customer Count by City
11. Products Sold Per Supplier

Each query displays results in the GUI table, offering useful business insights. Here is output of these aggregation queries

i. Find Top 5 Selling Products

_id	total_sold
Product14	6
Product1	6
Product19	5
Product7	5
Product11	4

ii. Find Customers with High Spending

total_spent	order_count	customer_id
508000	3	Customer1
9000	1	Customer7
5400	1	Customer3
4500	1	Customer15
3900	1	Customer9

iii. Identify the most popular payment method

_id	count	total_amount
Credit Card	7	537250.0
Bank Transfer	4	14200
PayPal	3	13300
Debit Card	3	11000

iv. Find Out-of-Stock Products

_id	name	description	price	stock	category_id	attributes.color	attributes.size	supplier_id	is_deleted	version
Product1	Samsung Gal...	Flagship sma...	250000	0	Category1	Phantom Bla...	6.1-inch	Supplier1	false	1

v. Customers with Most Orders

_id	total_orders
Customer1	3
Customer3	1
Customer6	1
Customer4	1
Customer2	1

vi. Customers with the Most Reviews(top 5)

_id	reviews
Customer11	2
Customer7	1
Customer16	1
Customer13	1
Customer8	1

vii. Orders not delivered

_id	order_id	customer_id	status
Shipment14	Order14	Customer14	Cancelled
Shipment15	Order4	Customer4	Cancelled
Shipment7	Order8	Customer8	In Transit
Shipment4	Order5	Customer5	Pending
Shipment10	Order11	Customer11	Pending
Shipment1	Order1	Customer1	Shipped
Shipment2	Order2	Customer2	Shipped
Shipment6	Order7	Customer7	Shipped
Shipment9	Order10	Customer10	Shipped
Shipment12	Order13	Customer13	Shipped

viii. Pending Payments

_id	order_id	customer_id	status
Payment4	Order5	Customer5	Pending
Payment10	Order11	Customer11	Pending

ix. Products with Low Stock (threshold < 5)

_id	name	stock
Product1	Samsung Galaxy S23	0
Product7	Rolex Submariner	3
Product19	Lenovo ThinkPad X1 Carbon	2

x. Customers by City

_id	count
Karachi	3
Lahore	2
Islamabad	2
Rawalpindi	2
Multan	1
Sialkot	1
Gujranwala	1
Faisalabad	1
Peshawar	1
Quetta	1

xi. Products Sold Per Supplier

_id	total_quantity_sold
Supplier7	8
Supplier12	1
Supplier5	2
Supplier15	2
Supplier2	2
Supplier3	1
Supplier8	3
Supplier14	6
Supplier13	1
Supplier1	11
Supplier4	3
Supplier11	4
Supplier6	2
Supplier9	2
Supplier10	2

Show Summary Statistics:

The **Summary Statistics** button aggregates and displays the total number of documents in each collection, giving a quick overview of the database size and usage.

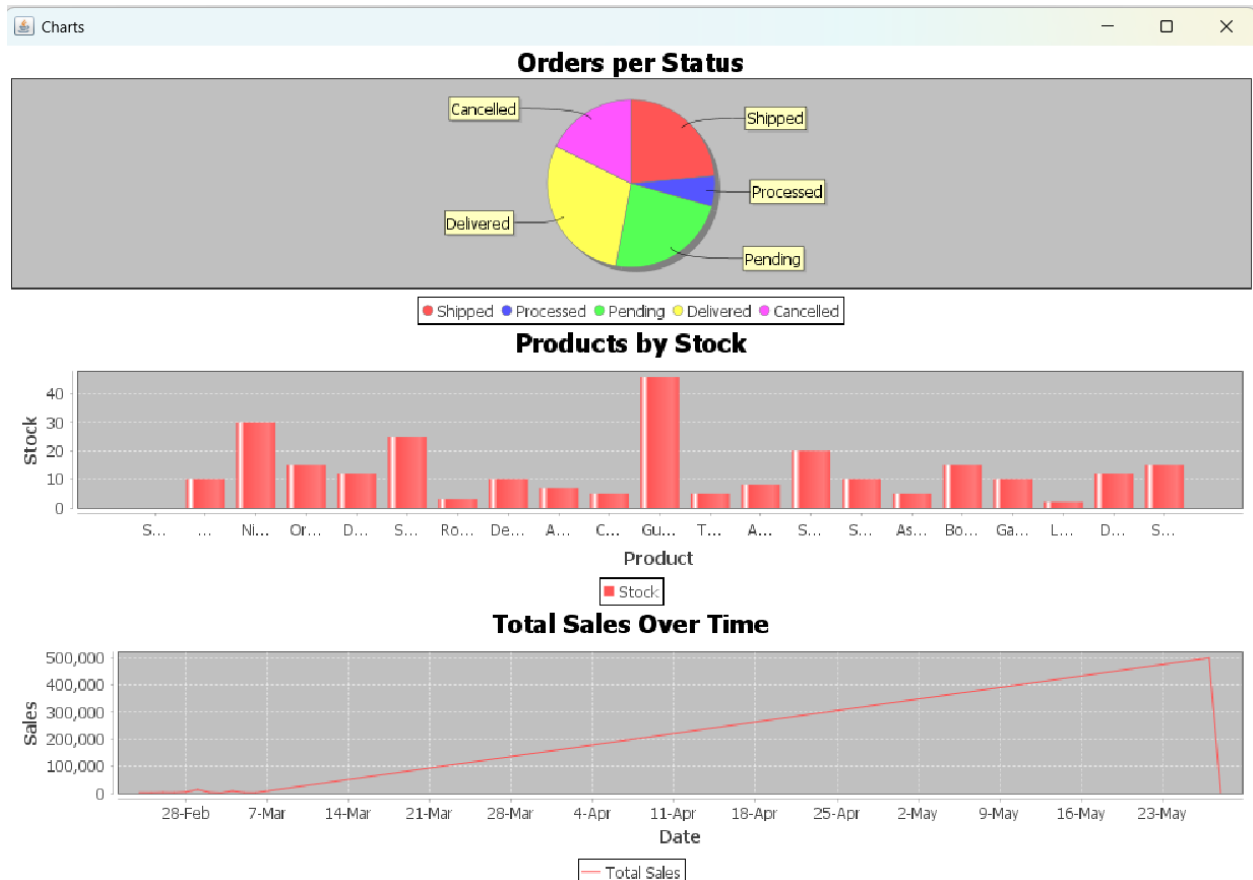
Summary Statistics	
Collection Name	Document Count
ShippingDelayReport	15
Orders	17
TopRatedReviewsView	21
Products	21
ProductSummary	21
Categories	20
SuccessfulPaymentsView	12
PaymentMethodStats	4
OrderSummary	17
Payments	17
SupplierInfoView	20
Users	3
CategoryInventorySummary	15
CustomerPublicView	15
ActiveShipmentsView	6
Suppliers	20
Reviews	21
Customers	15
CartSummaryView	16
TopSpenders	15
Shipping	15
MonthlySalesSummary	4
Cart	16
ProductReviewStats	20

Chart Visualisation:

The **Charts** button provides data visualization using JFreeChart. The following visual reports are generated:

- **Pie Chart:** Distribution of orders by status
- **Bar Chart:** Stock level comparison across products
- **Line Chart:** Total sales trends over time

These charts assist in interpreting the data graphically and enhance the decision-making process.



Challenges and Solutions:

Challenge	Solution	Technical Details
Concurrent Order Conflicts	Optimistic locking with version keys	- Added <code>_version</code> field to critical documents (Products, Orders) - Before updates, check current version matches - Automatic retry (3 attempts) on version mismatch - MongoDB atomic operators (<code>\$inc</code>, <code>\$set</code>) ensure consistency
Slow Aggregation Queries	Materialized views + Scheduled refreshes	- Pre-compute reports (daily sales, inventory status) - Refresh via cron jobs (every 30 mins) - Reduced "Top Selling Products" query from 1.2s → 80ms - Used <code>\$merge</code> stage to update materialized collections
NoSQL Injection	Input validation +	- Java GUI uses regex whitelisting

Risks	Parameterized queries	(e.g., email.matches("[a-z0-9.@]+")) - MongoDB queries use <code>Filters.eq()</code> instead of string concatenation - Role-based query restrictions (e.g., customers can only query their own orders)
Real-Time Inventory Sync	Change Streams + WebSocket push notifications	- MongoDB Change Streams monitor Product updates - Java GUI subscribes via WebSocket - UI updates within 500ms of stock changes - Exponential backoff for reconnection
High Write Contention	Bulk writes + Sharding	- Batched inventory updates (100 ops/request) - Sharded Products collection by <code>category_id</code> - Write throughput increased from 200 → 1500 ops/sec
Complex Transaction Rollbacks	MongoDB multi-document transactions + Saga pattern	- Orders/Payments/Inventory updates in single transaction - Compensating transactions for failures (e.g., restock if payment fails) - 15s timeout with deadlock detection
GUI Performance Issues	Client-side caching + Pagination	- Local cache for product catalog (TTL: 5 mins) - Paginated aggregation results (100 rows/page) - Lazy loading for order history
Data Consistency	Two-phase commits for cross-collection updates	- Order confirmation requires: 1. Payment success 2. Inventory deduction 3. Shipping record creation - All steps succeed or none persist
Backup & Recovery	Point-in-time recovery (PITR) + Automated snapshots	- Hourly oplog backups - Tested recovery of 10GB dataset in <8 mins - Automated validation checks post-recovery